

AI-Powered E-Commerce Cloud Architecture Report

This report explains the complete architecture, deployment process, networking flow, AI integration, debugging challenges, and cloud infrastructure of the AI-powered E-commerce platform deployed on AWS.

1. Project Overview

The project is a production-grade AI-powered E-commerce platform deployed entirely on AWS cloud infrastructure. The platform combines cloud computing, Docker, microservices, machine learning, vector databases, and semantic AI search.

2. Core Technologies Used

Frontend: React
Backend: NestJS
AI Service: FastAPI
Database: PostgreSQL + pgvector
Cloud Provider: AWS
Machine Learning: HuggingFace all-MiniLM-L6-v2
AI Inference: AWS SageMaker
Containerization: Docker & Docker Compose

3. Final Cloud Architecture

User Browser → AWS S3 → AWS EC2 Backend → Docker Bridge Network → FastAPI AI Service → AWS SageMaker → AWS RDS PostgreSQL + pgvector

4. Frontend Hosting Using AWS S3

The React frontend was compiled into static assets and uploaded to an AWS S3 bucket configured for static website hosting. S3 acts as a globally optimized storage and delivery system that serves HTML, CSS, and JavaScript directly to users without requiring a running server.

5. Backend Deployment on EC2

The NestJS backend was containerized using Docker and deployed to an AWS EC2 virtual machine. The backend handles authentication, API routing, business logic, product management, and communication with the AI microservice.

6. Docker Microservice Architecture

Two containers run side-by-side on the EC2 instance:

- nexshop-backend (NestJS Backend)
- nexshop-ai (FastAPI AI Service)

Both containers communicate internally through a secure Docker bridge network called nexshop-network.

7. Internal Docker Networking

The backend communicates with the AI service using the internal Docker DNS system. Instead of calling a public IP address, the backend directly calls:

`http://ai:8000`

Docker automatically resolves the hostname 'ai' to the internal container IP address. This avoids exposing the AI service publicly and improves security and performance.

8. AI Semantic Search Flow

When a user types a search query such as 'gming', the request flows through the following stages:

1. React frontend sends request to backend
2. Backend forwards request to AI container
3. AI container sends text to SageMaker
4. SageMaker generates a 384-dimensional embedding vector
5. AI service queries PostgreSQL using pgvector
6. Database calculates cosine similarity
7. Best matching products are returned

9. AWS SageMaker Integration

Instead of hosting the NLP model locally on EC2, the model was deployed to AWS SageMaker. The FastAPI AI service uses boto3 to securely invoke the SageMaker endpoint and retrieve semantic embeddings.

10. Vector Database Using pgvector

PostgreSQL was extended using the pgvector extension. Each product description was converted into a 384-dimensional mathematical vector representation and stored permanently in the database. Semantic search is performed using cosine similarity calculations through the PostgreSQL vector operator (`<=>`).

11. Challenge: ISP Port 80 Interception

One of the major deployment issues occurred because Telecom Egypt intercepted HTTP traffic on Port 80 after the internet quota expired. Instead of returning JSON API responses, the ISP injected HTML warning pages into the API responses.

Solution:

The backend was migrated from Port 80 to Port 3000, Docker mappings were updated, EC2 security groups were modified, and frontend API URLs were changed accordingly.

12. Challenge: AWS RDS SSL Failures

AWS RDS enforces encrypted SSL/TLS database connections by default. Initially, the backend attempted a standard TCP connection which was rejected.

Solution:

SSL support was added to the Drizzle ORM PostgreSQL connection configuration using: `ssl: { rejectUnauthorized: false }`

13. Challenge: SageMaker IAM Restrictions

AWS Academy Learner Labs prevented creation of custom IAM roles. Because of this restriction, the EC2 instance could not directly access SageMaker.

Solution:

Temporary AWS credentials were manually injected into the Docker container environment variables.

14. Challenge: NumPy Inhomogeneous Array Crash

The AI service initially crashed because SageMaker returned variable-length token embeddings. NumPy failed when attempting to combine arrays with different dimensions.

Solution:

NumPy was removed entirely. The parsing logic was rewritten using pure Python lists, specifically extracting the [CLS] token embedding to ensure a fixed 384-dimensional vector.

15. Challenge: PostgreSQL Vector Type Failure

After removing NumPy, psycopg2 formatted vectors incorrectly as double precision arrays. PostgreSQL rejected cosine similarity calculations because pgvector requires the vector type.

Solution:

Vectors were manually formatted as string literals before being passed into SQL queries. PostgreSQL automatically parsed the strings into native vector objects.

16. Production Initialization

A raw Python script was executed inside the live Docker container using `docker exec`. The script:

- Installed the `pgvector` extension
- Generated embeddings for all products
- Stored semantic vectors permanently in the RDS database

17. Key Cloud Computing Concepts Demonstrated

This project demonstrates:

- Cloud deployment
- Docker containerization
- Microservice architecture
- AI inference systems
- Vector databases
- Docker networking
- Semantic search
- AWS infrastructure management
- SSL/TLS encryption
- Production debugging

18. Final Conclusion

This project evolved from a standard E-commerce platform into a complete AI-powered cloud computing system. The final architecture demonstrates advanced engineering concepts including distributed systems, semantic vector search, machine learning inference, Docker networking, AWS infrastructure deployment, and production-grade troubleshooting.

The platform successfully handles intelligent AI product search in under 400 milliseconds, even when users make spelling mistakes or search using contextual intent.